File-Layout-Aware Caching for Cloud-Native Analytical Query Engines

Samuel Khuon
Advisor: Ilya Meignan--Masson
Systems Research Group
https://dse.in.tum.de/

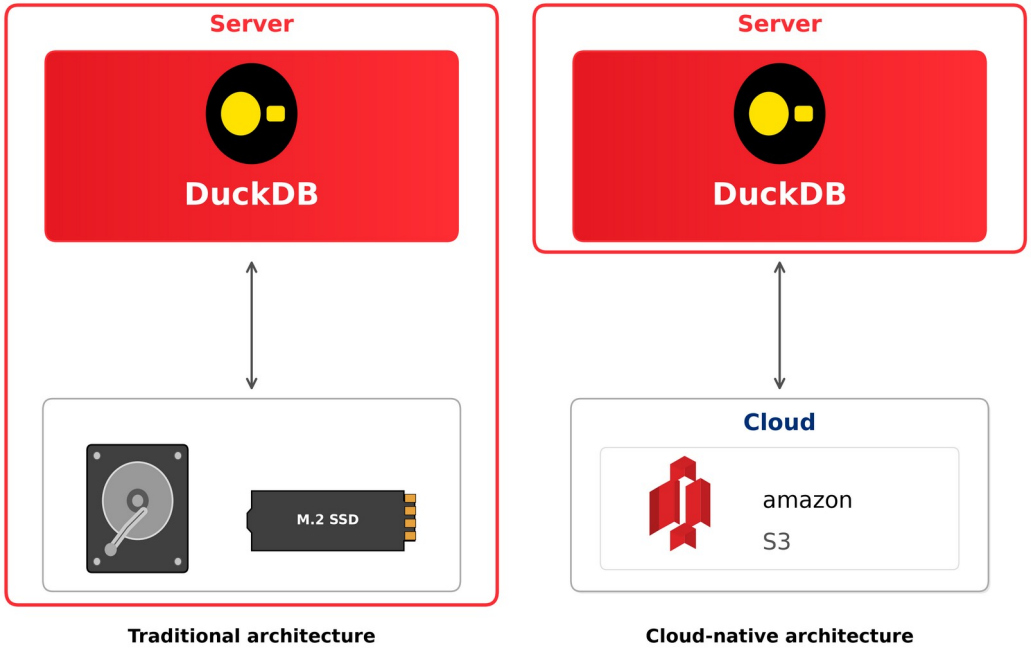


27.10.2026

Background

Cloud-native database architecture:

- separates compute and storage
- greater flexibility
- independent scalability



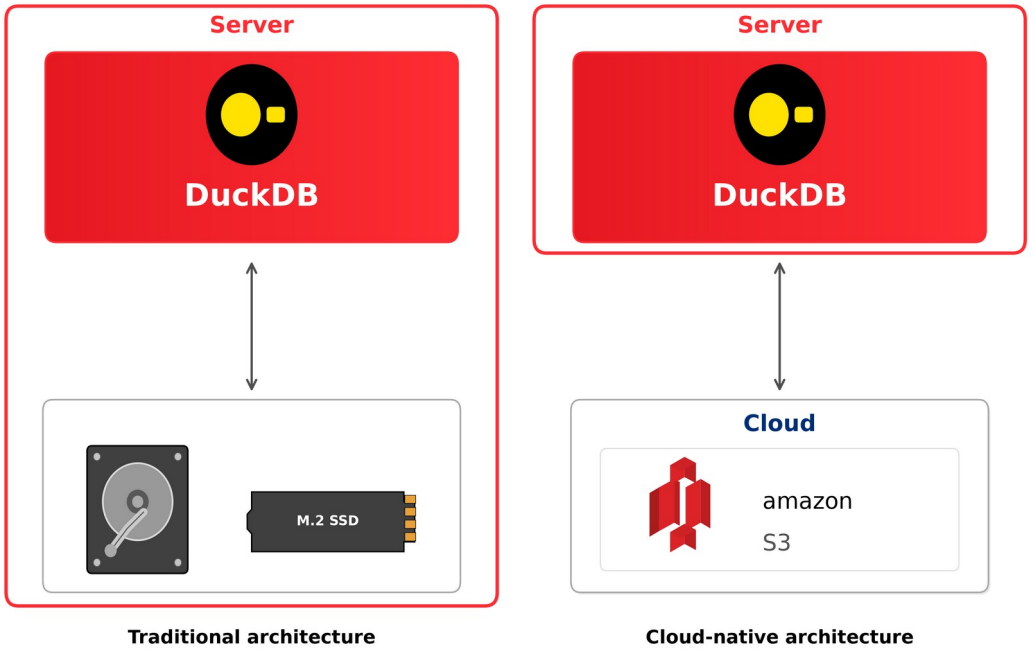
Traditional architecture

Cloud-native architecture

Background

Cloud-native database architecture:

- separates compute and storage
- greater flexibility
- independent scalability



Traditional architecture

Cloud-native architecture

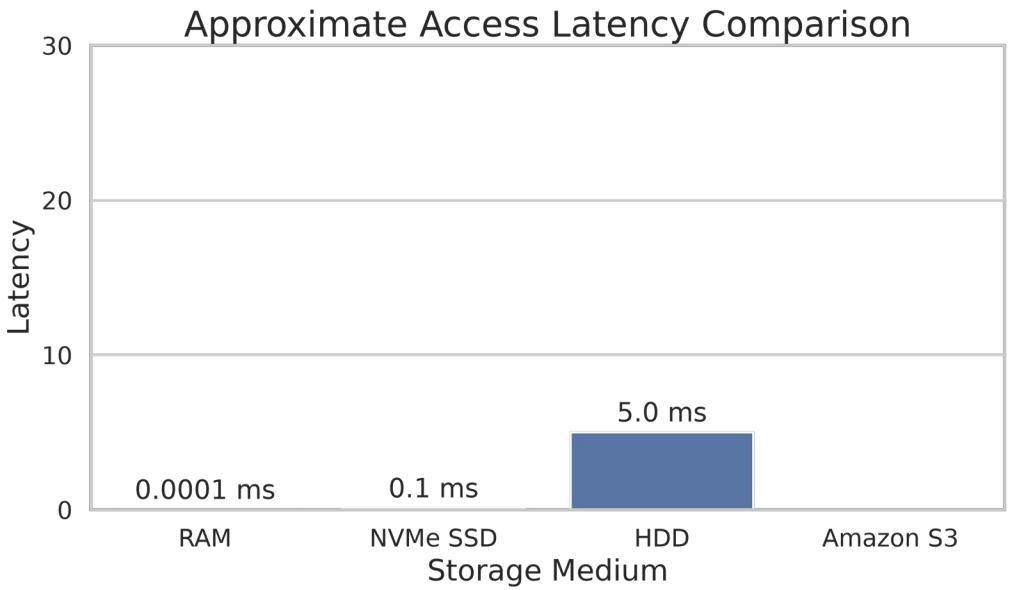
What new challenges does this architecture introduce?

New Challenges

Remote storage has distinct performance characteristics:

+ high Bandwidth

- high Latency

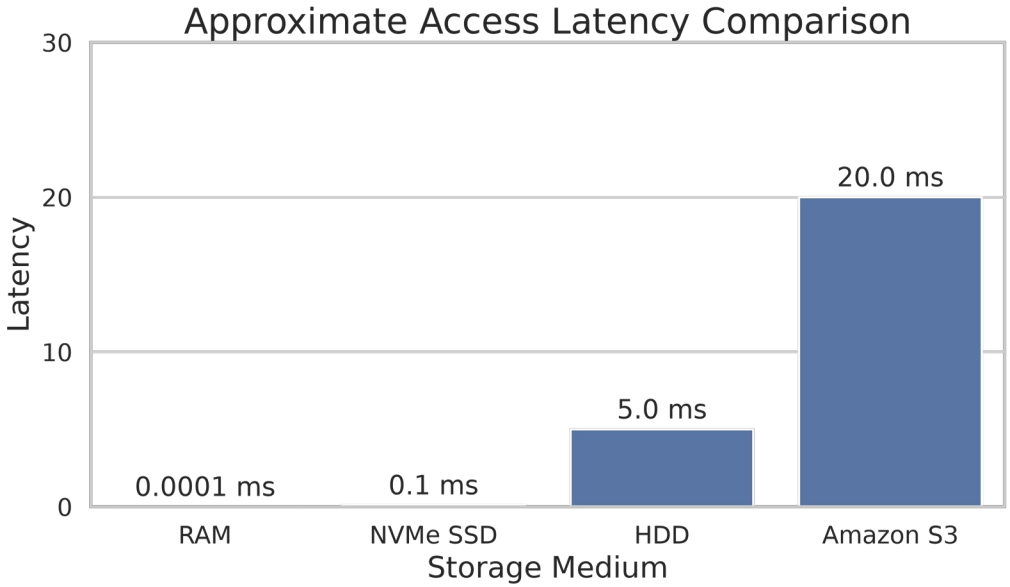


New Challenges

Remote storage has distinct performance characteristics:

+ high Bandwidth

- high Latency

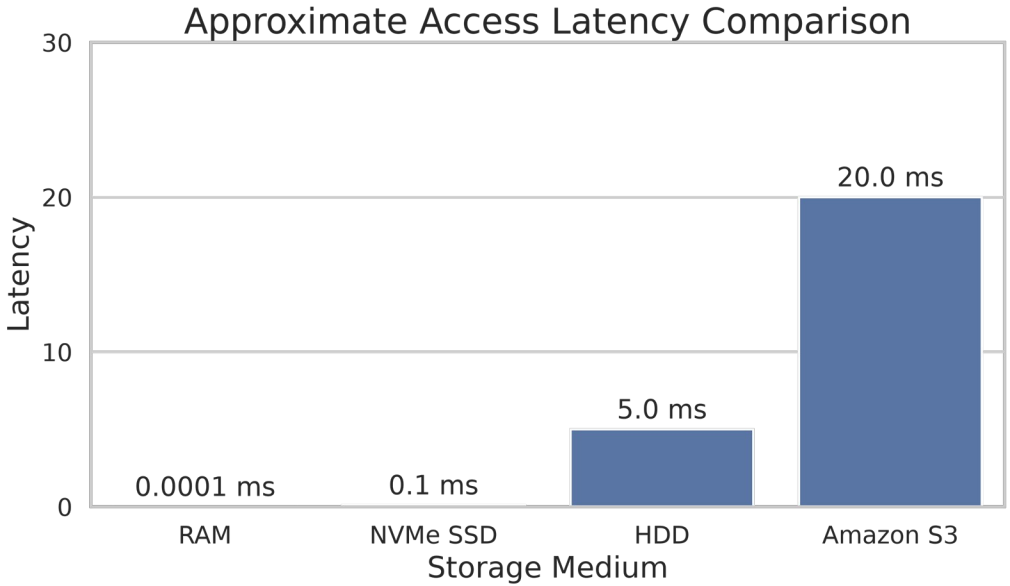


New Challenges

Remote storage has distinct performance characteristics:

+ high Bandwidth

- high Latency

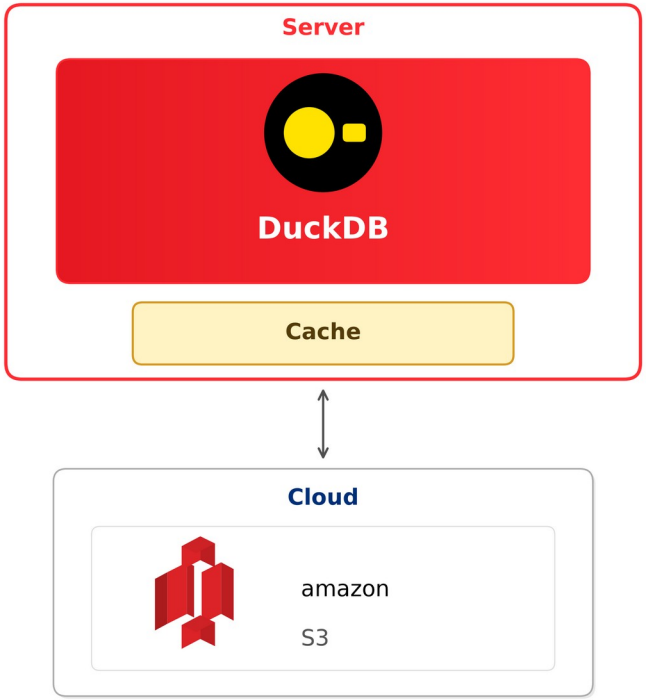


How can we reduce accesses to persistent storage?

Research Gap

Traditional caches are typically general-purpose

- Unaware of **file layout**
- Unaware of **storage performance**
- Use **generic replacement policies**



Problem Statement

How can we design a cache optimized for analytical workloads in cloud environments?

Solution

A cache tailored to how cloud analytics data is structured, stored, and accessed.

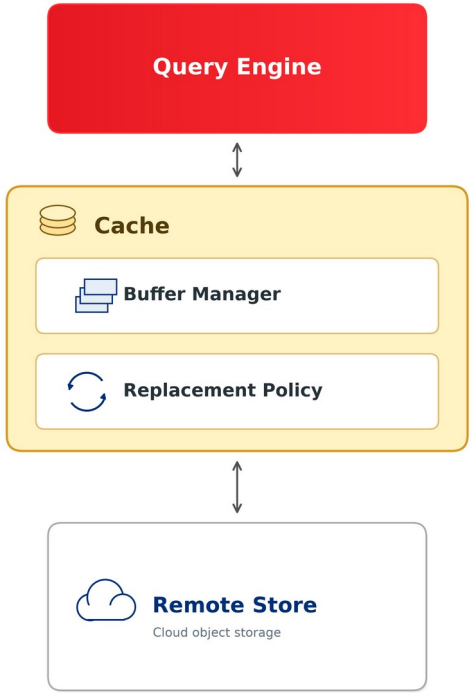
Leverage …

file layout
Storage performance characteristics
analytic access patterns

Overview

Buffer Manager
- Dynamically sized blocks

Replacement Policy
- Workload-adapted
- Size-aware



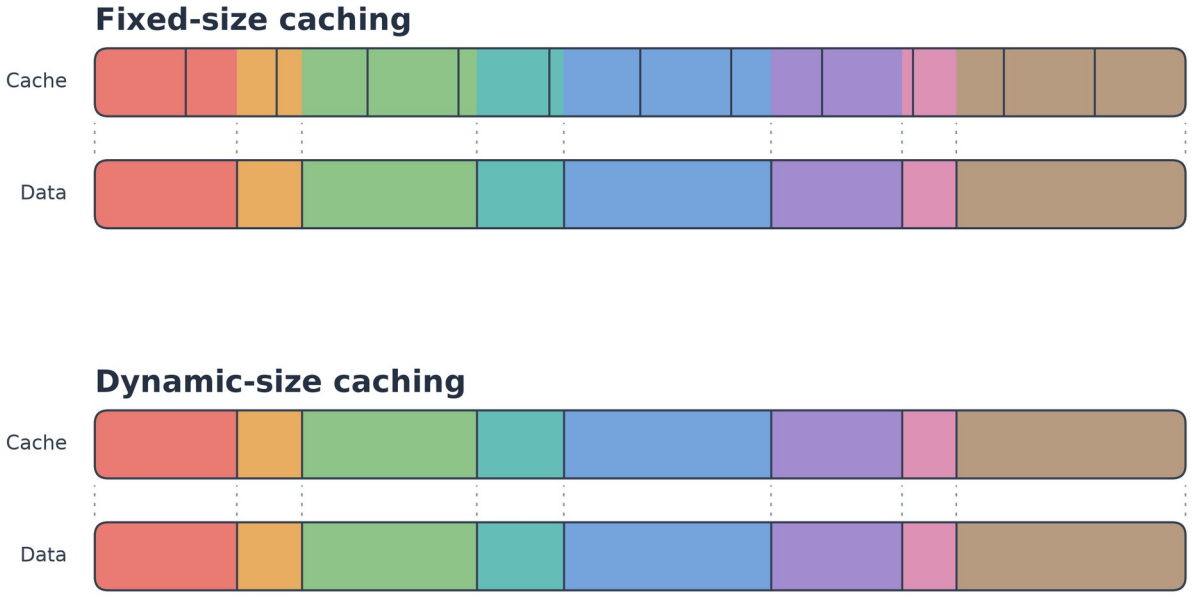
Dynamic Blocks

Problem: Unaware of file structure

- unused data fetched
- unused data stored

Solution: Caching based on logical blocks of file format

+ less data fetched
+ less data stored

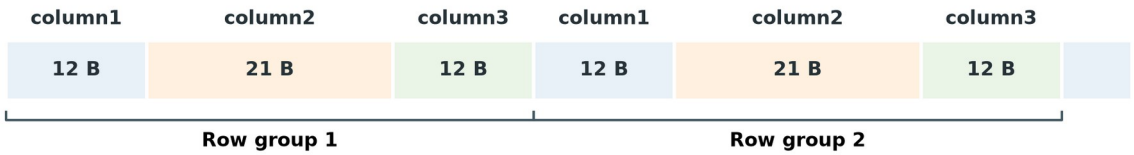


Dynamic Blocks

Table → columnar file

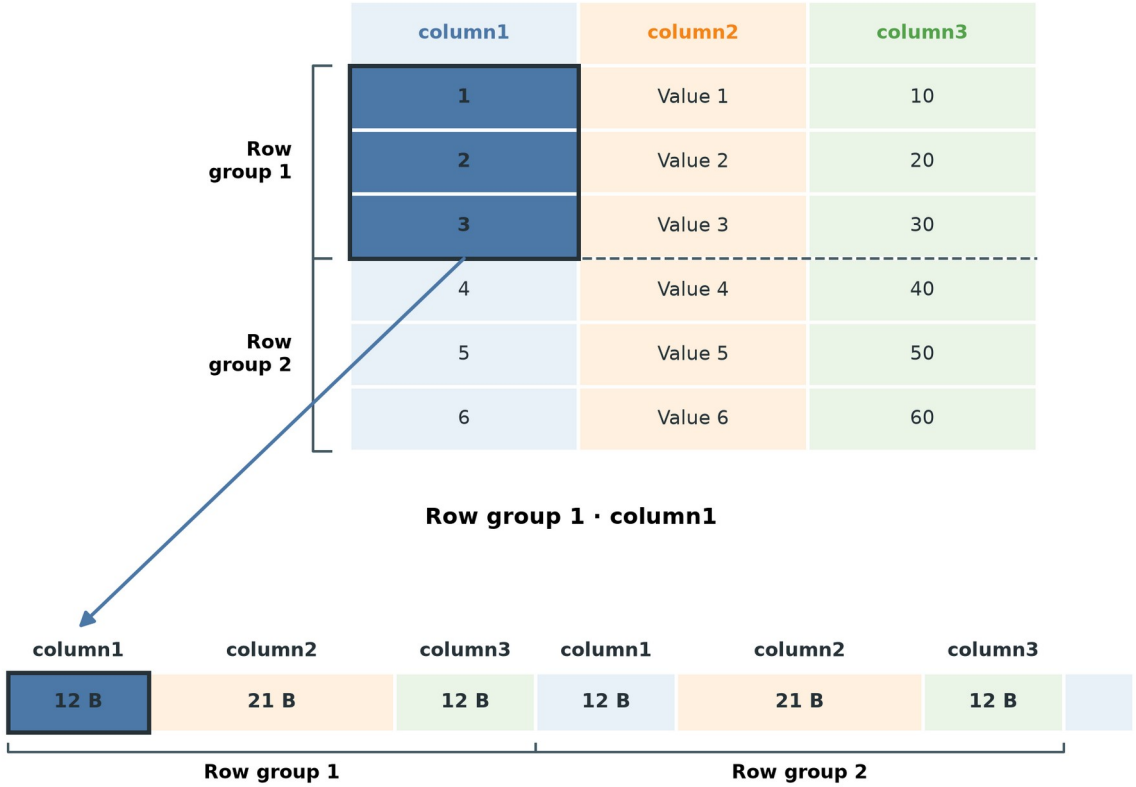
	column1	column2	column3
Row group 1	1	Value 1	10
	2	Value 2	20
	3	Value 3	30
Row group 2	4	Value 4	40
	5	Value 5	50
	6	Value 6	60

One chunk per column, per row group



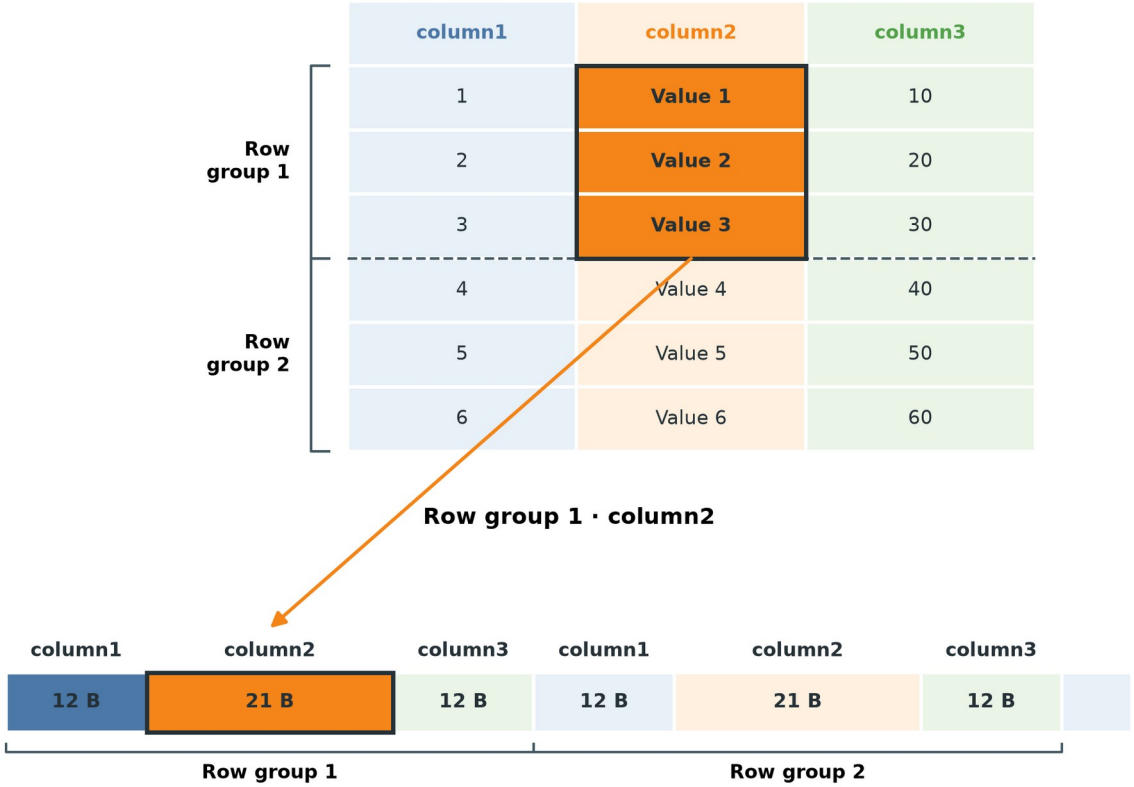
Dynamic Blocks

Table → columnar file



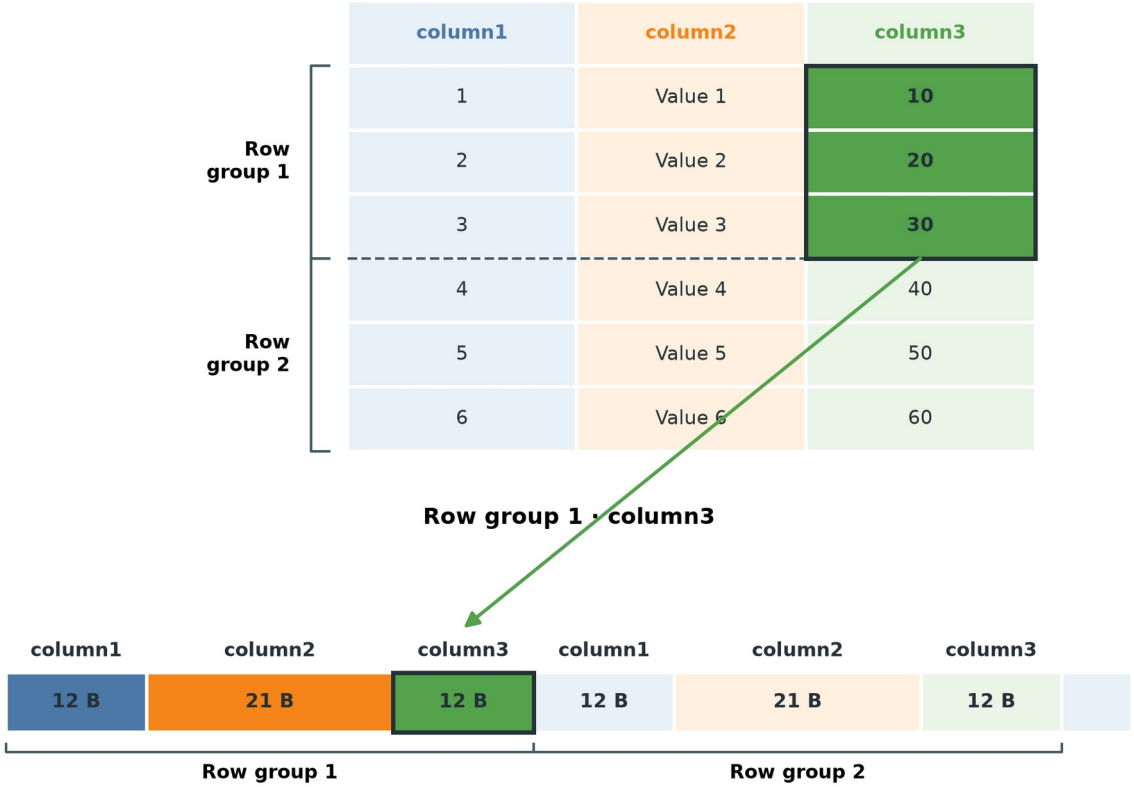
Dynamic Blocks

Table → columnar file



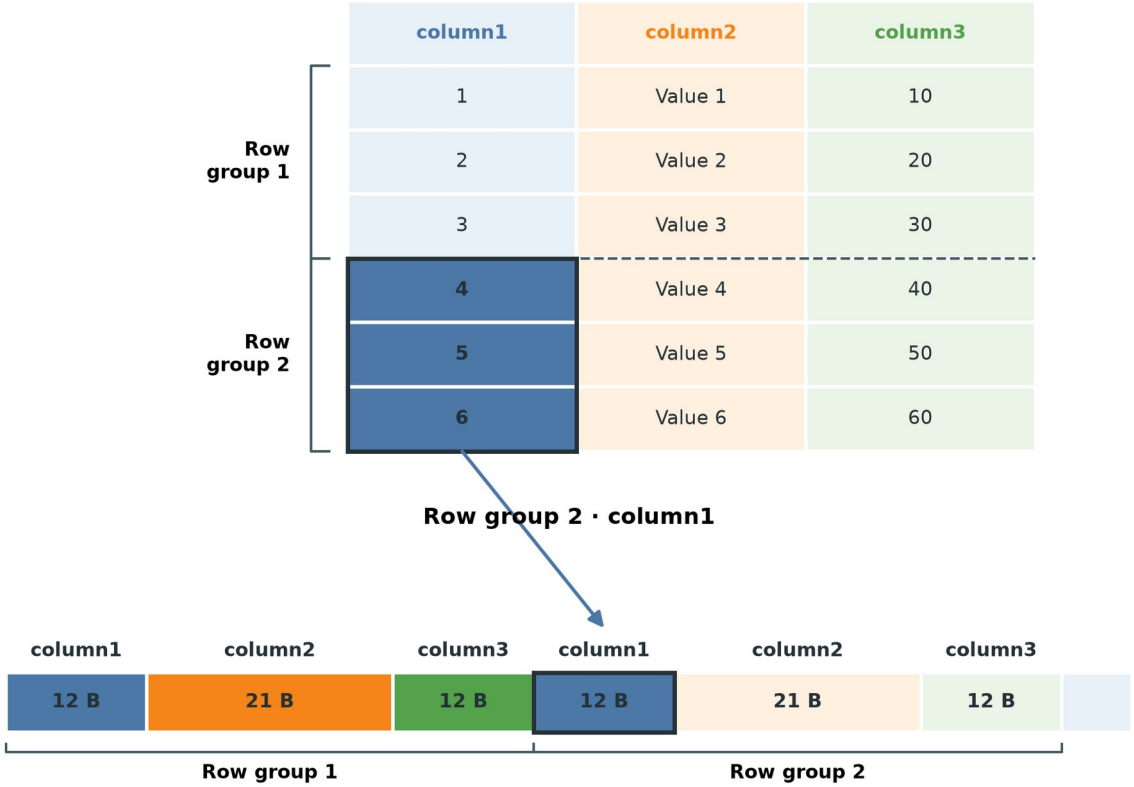
Dynamic Blocks

Table → columnar file



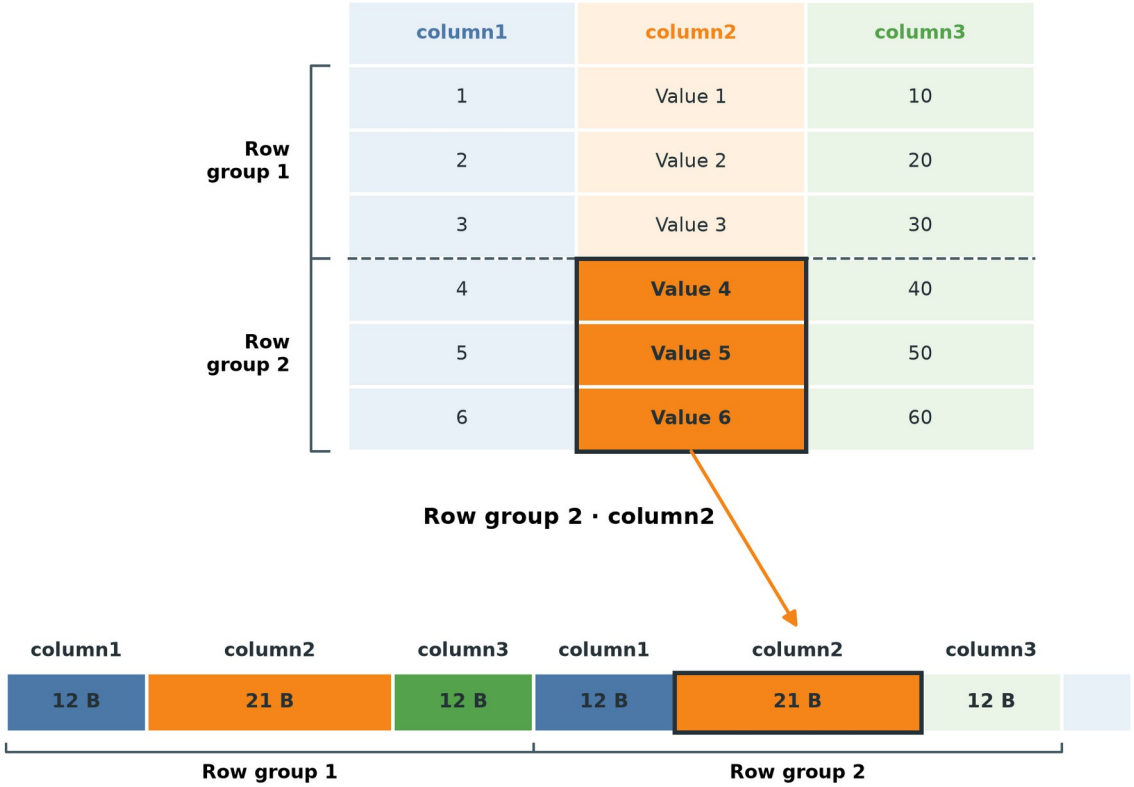
Dynamic Blocks

Table → columnar file



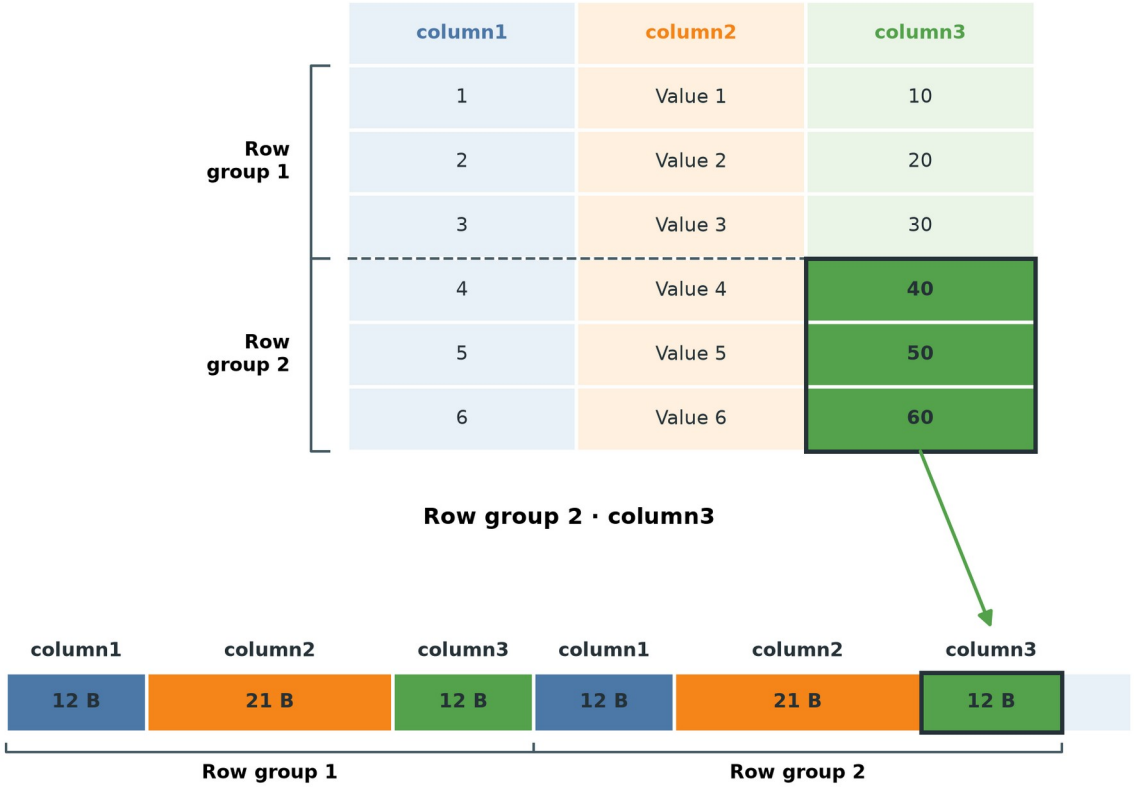
Dynamic Blocks

Table → columnar file



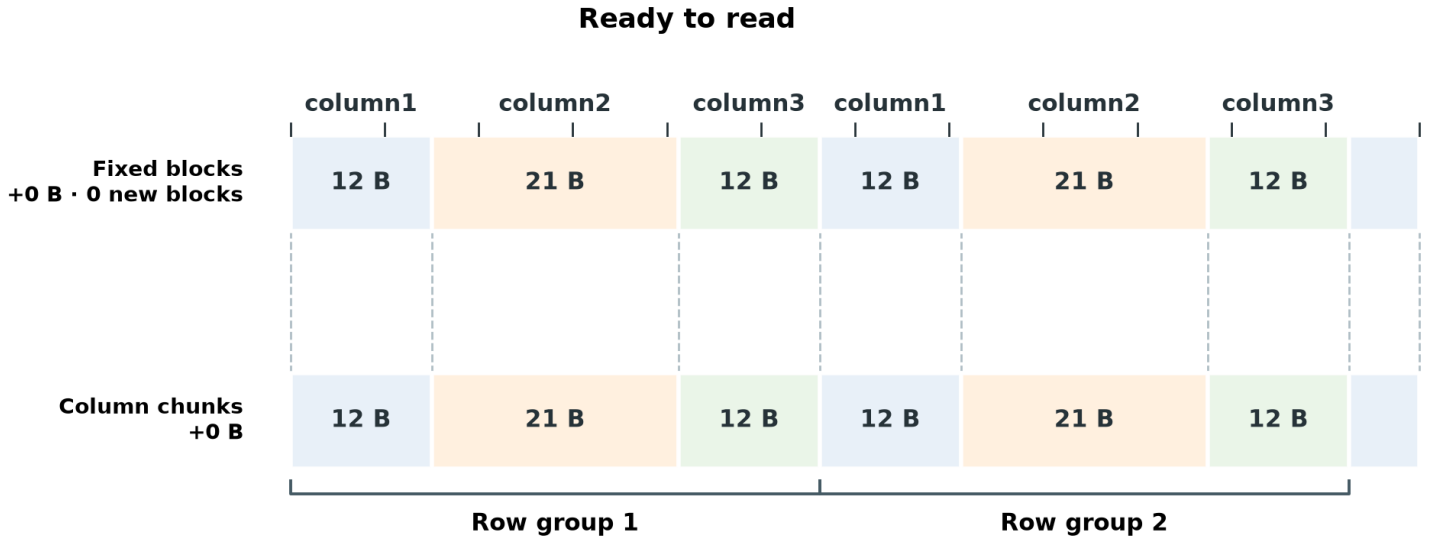
Dynamic Blocks

Table → columnar file



Dynamic Blocks

Read column1 + column3

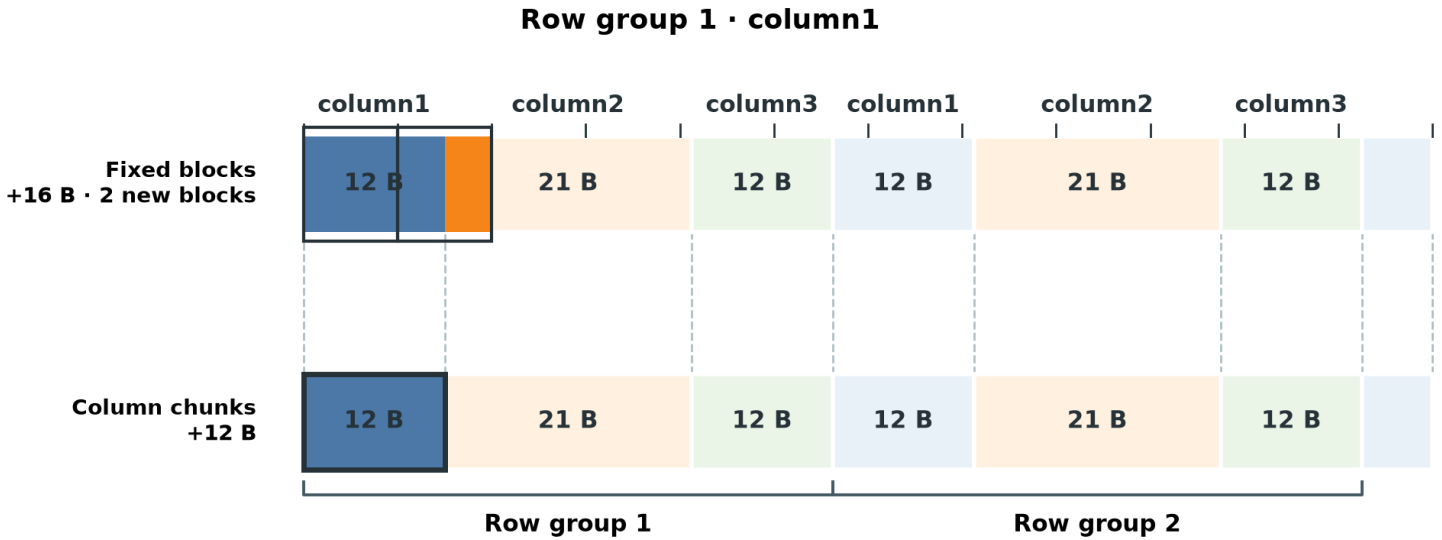


Total: fixed 0 B · chunks 0 B · extra -

Bright = loaded · Outline = new read · Blocks = 8 B

Dynamic Blocks

Read column1 + column3

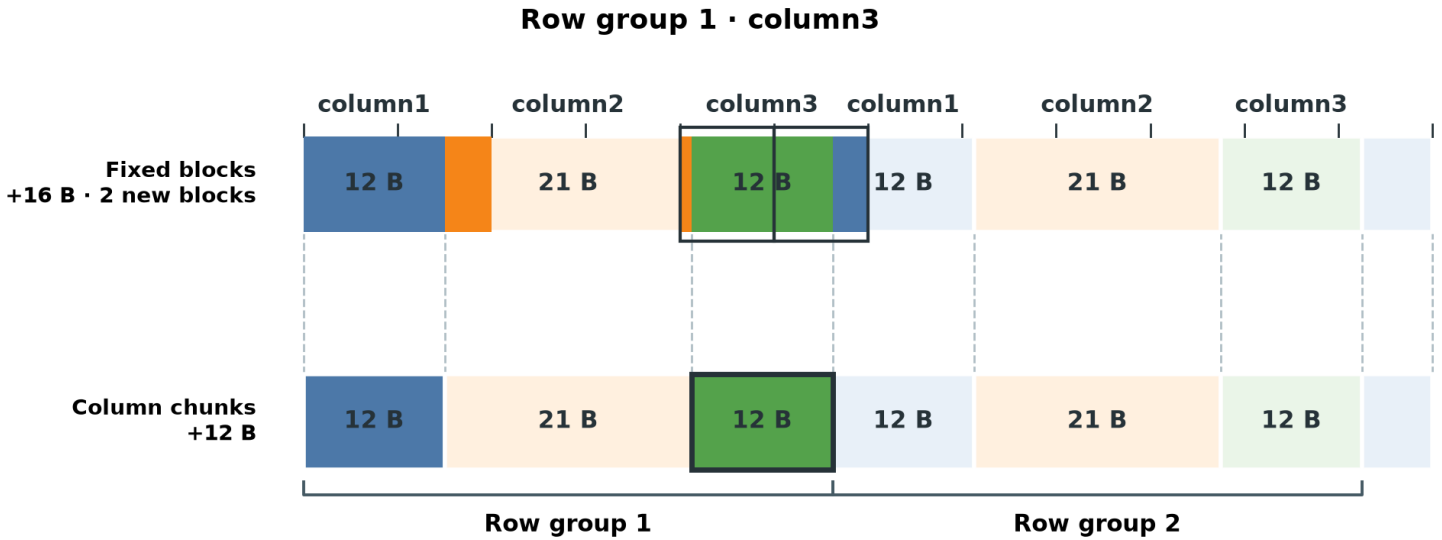


Total: fixed 16 B · chunks 12 B · extra 33%

Bright = loaded · Outline = new read · Blocks = 8 B

Dynamic Blocks

Read column1 + column3

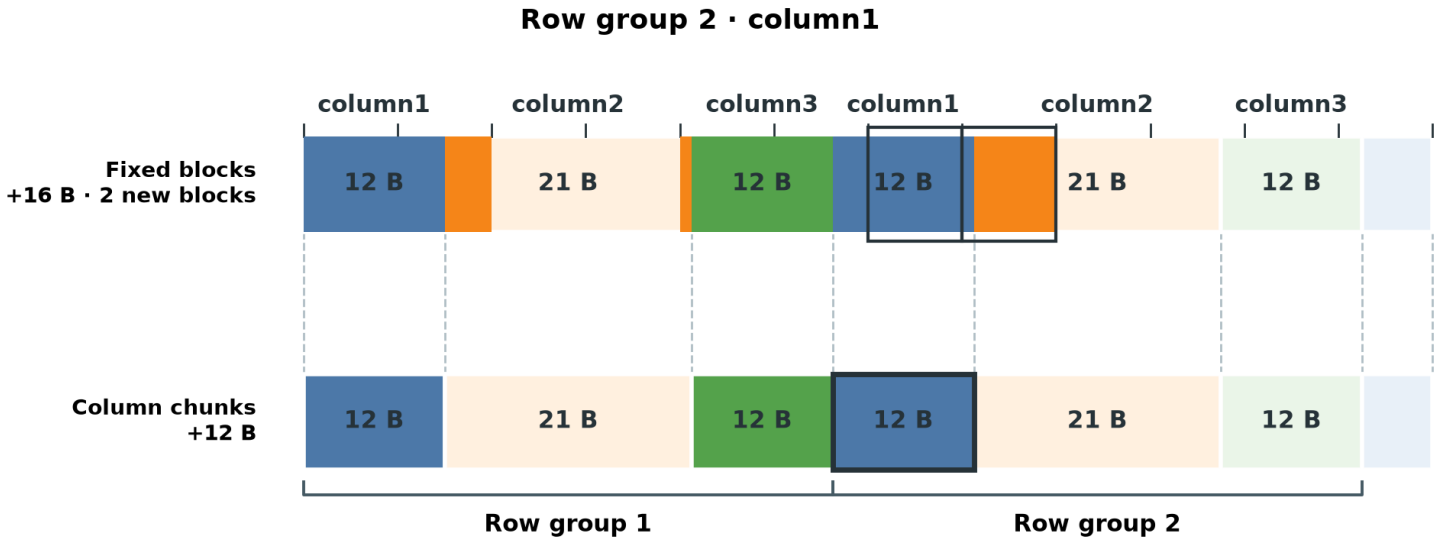


Total: fixed 32 B · chunks 24 B · extra 33%

Bright = loaded · Outline = new read · Blocks = 8 B

Dynamic Blocks

Read column1 + column3

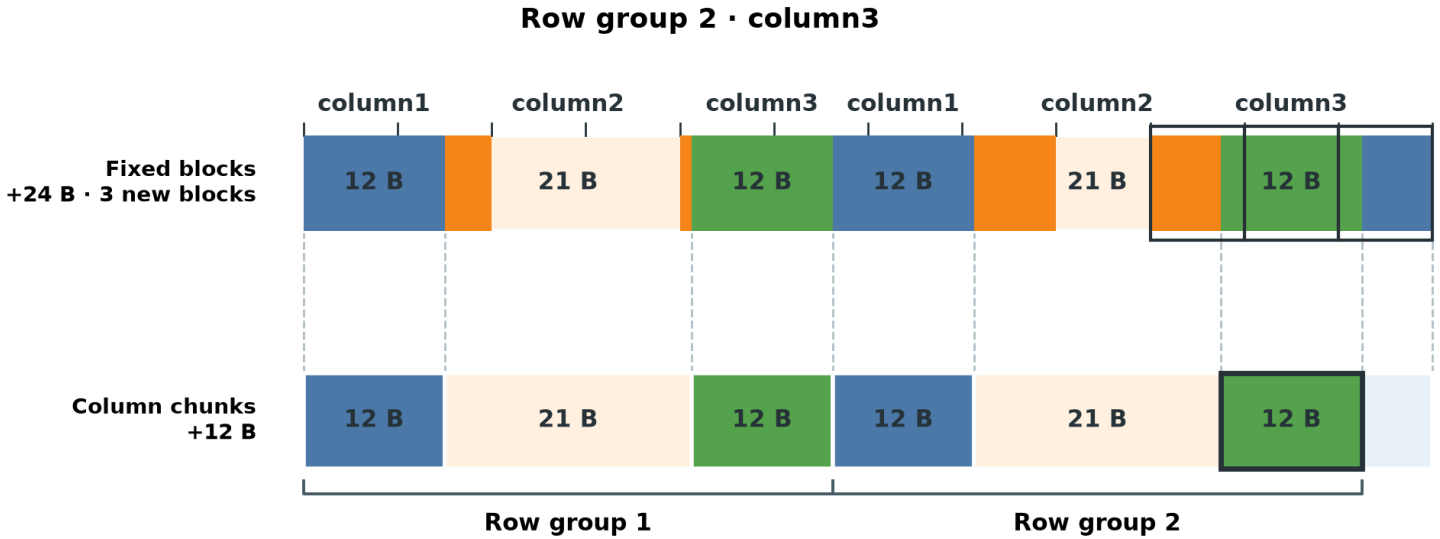


Total: fixed 48 B · chunks 36 B · extra 33%

Bright = loaded · Outline = new read · Blocks = 8 B

Dynamic Blocks

Read column1 + column3



Total: fixed 72 B · chunks 48 B · extra 50%

Bright = loaded · Outline = new read · Blocks = 8 B

Replacement Policy

Problem: Policies aren't optimized for analytical workloads

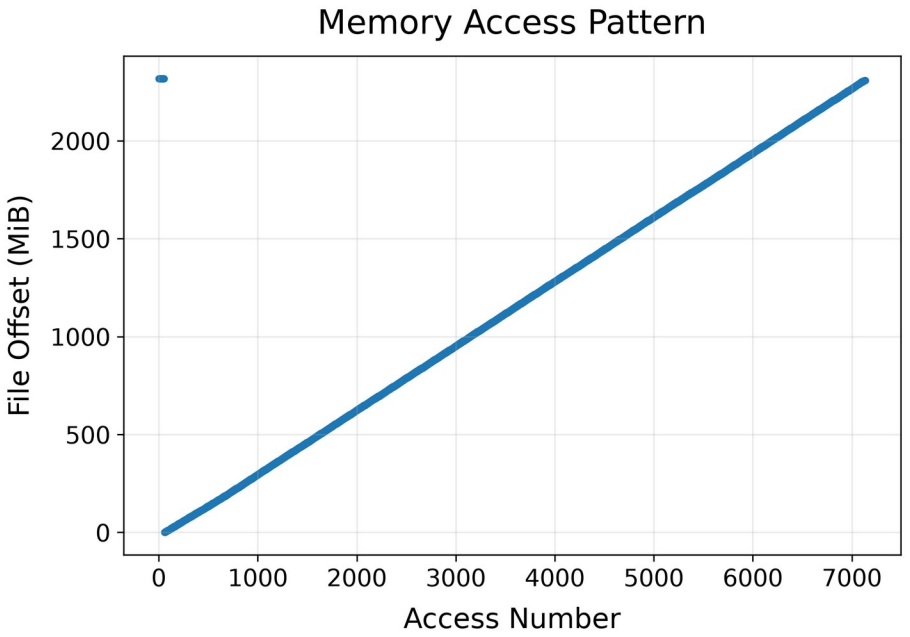
- cache pollution by long scans

- hot blocks displaced by single-use blocks

Solution: Scan-resistant policy

+ evict single-use blocks faster

+ keep high-reuse blocks longer

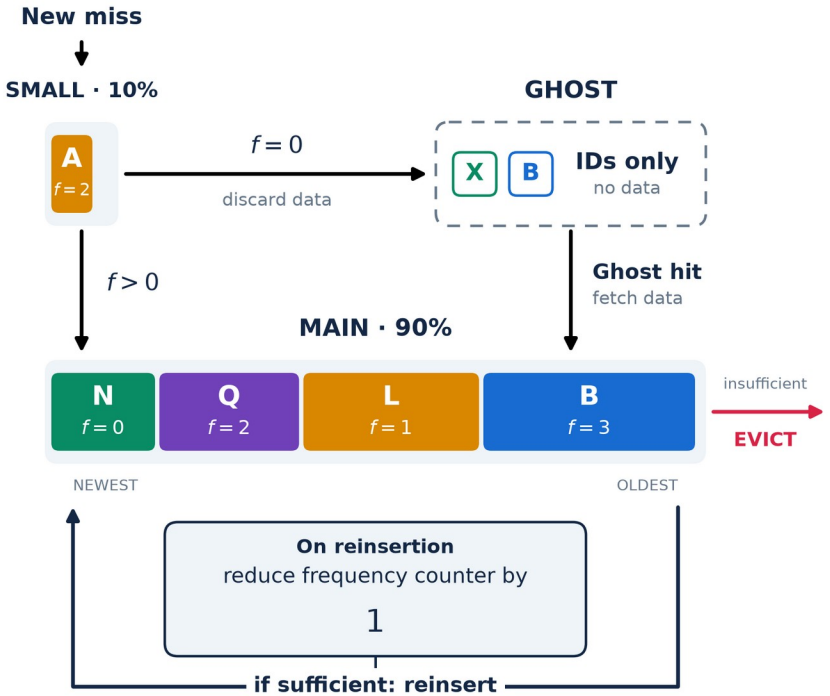


S3-FIFO [SOSP'24]

Simple, scan-resistant replacement policy

Divides the cache into three FIFO queues:

- Small: Holds new entries; promotes reused entries
- Main: Holds frequently reused entries
- Ghost: Tracks evicted entries to detect reuse



Size-Aware Replacement Policy

Problem: Blocks now have different sizes

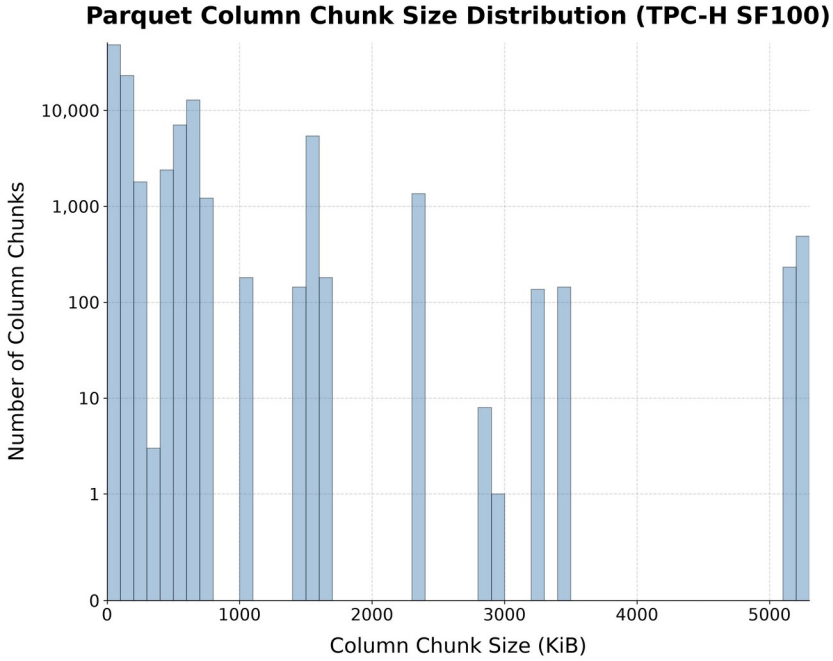
- blocks differ in storage and access cost

- equal treatment leads to suboptimal eviction

Solution: Size-aware replacement policy

+ eviction decision based on cost

+ cache utilized more efficiently



Size-aware S3-FIFO

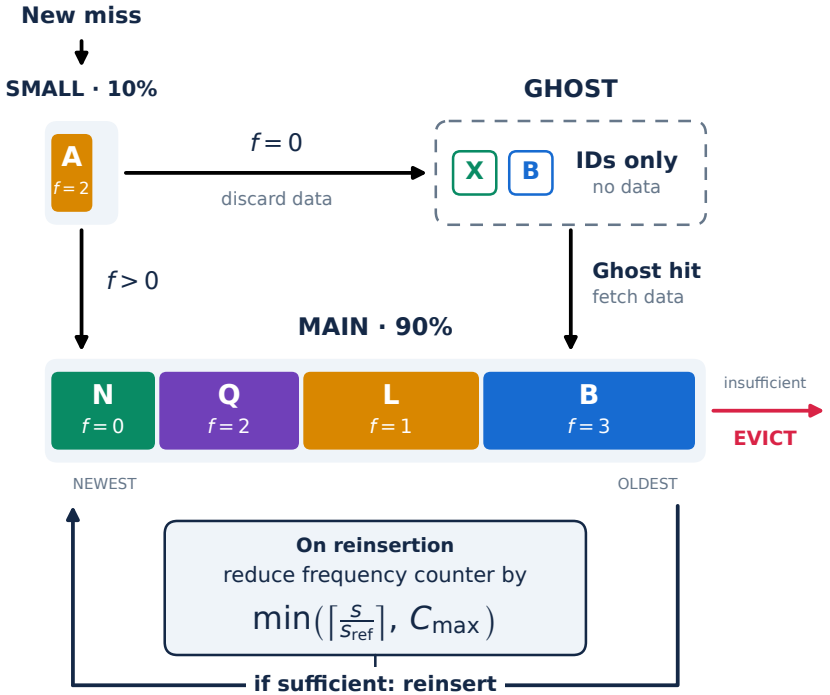
Small blocks: cheap to store but expensive to fetch due to fixed latency

Large blocks: consume more cache space but have lower relative fetch costs

Size-aware cost: $$C(s) = min\left(\left\lceil \frac{s}{s_{ref}} \right\rceil, C_{max}\right)$$

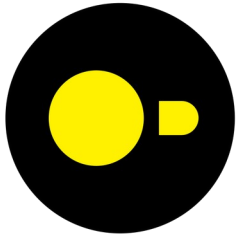
s_{ref} : reference block size

C_{max} : maximal cost



Implementation

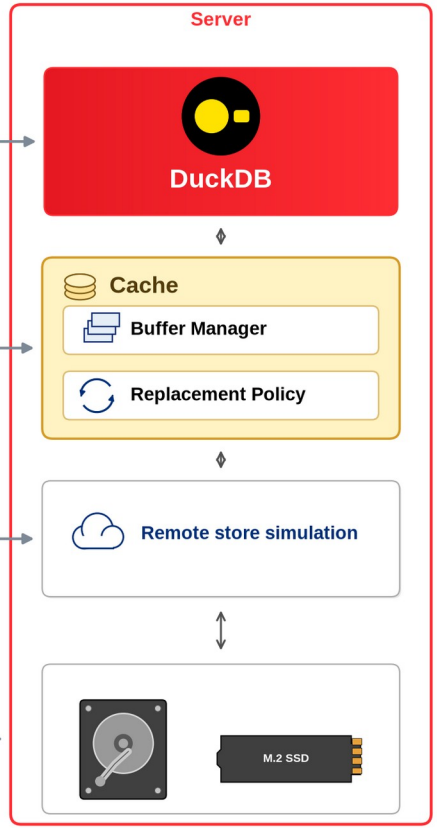
- Implemented for DuckDB, building upon community module
- Query engine read requests are mapped to Parquet column chunks.
- Buffer manager delegates eviction decisions through a generic policy interface
- LRU, S3-FIFO, and Size-aware S3-FIFO implement this interface



DuckDB

Evaluation Setup

Threads	64
Cache size	20% of dataset
Simulated storage	20 ms & 0.5 Gbit/s
Benchmark	TPC-H SF100 ~35GiB

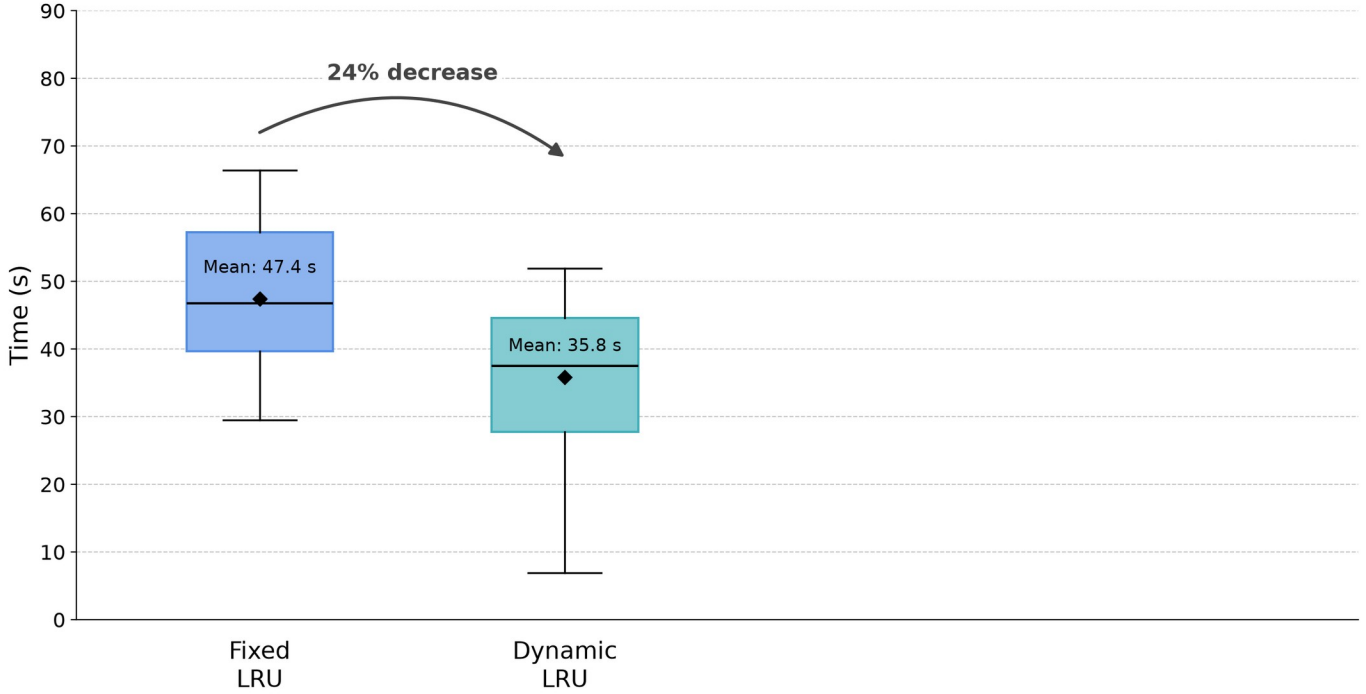


Dynamic Blocks



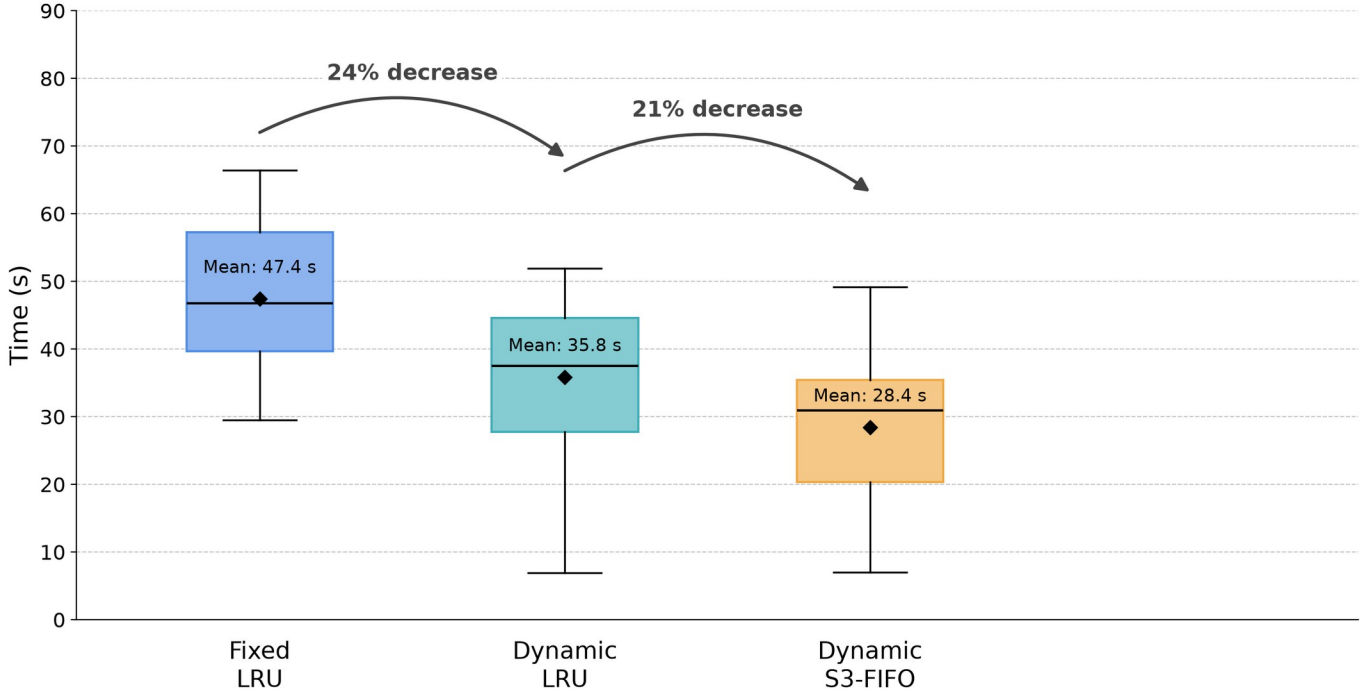
Benchmark: 20 sequences of 5 queries State:warm

Dynamic Blocks



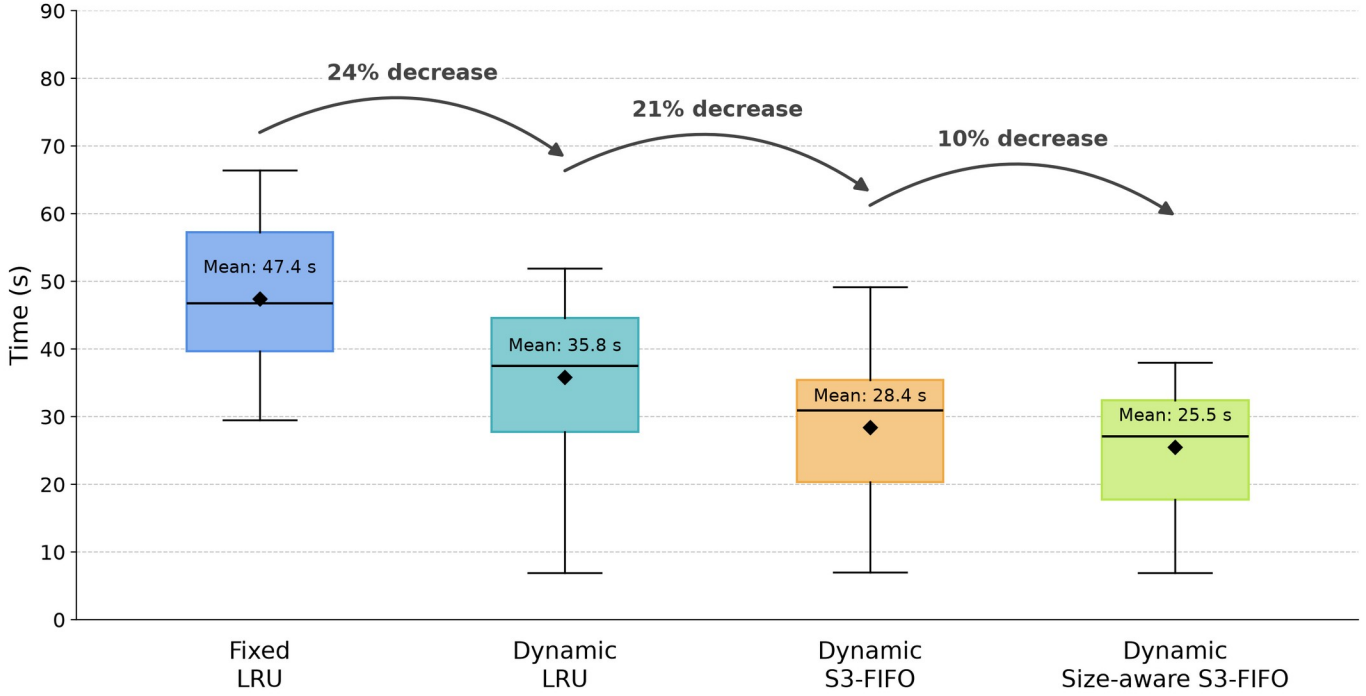
Sequences of queries also improve with dynamic block sizes

Scan-Resistant Policy



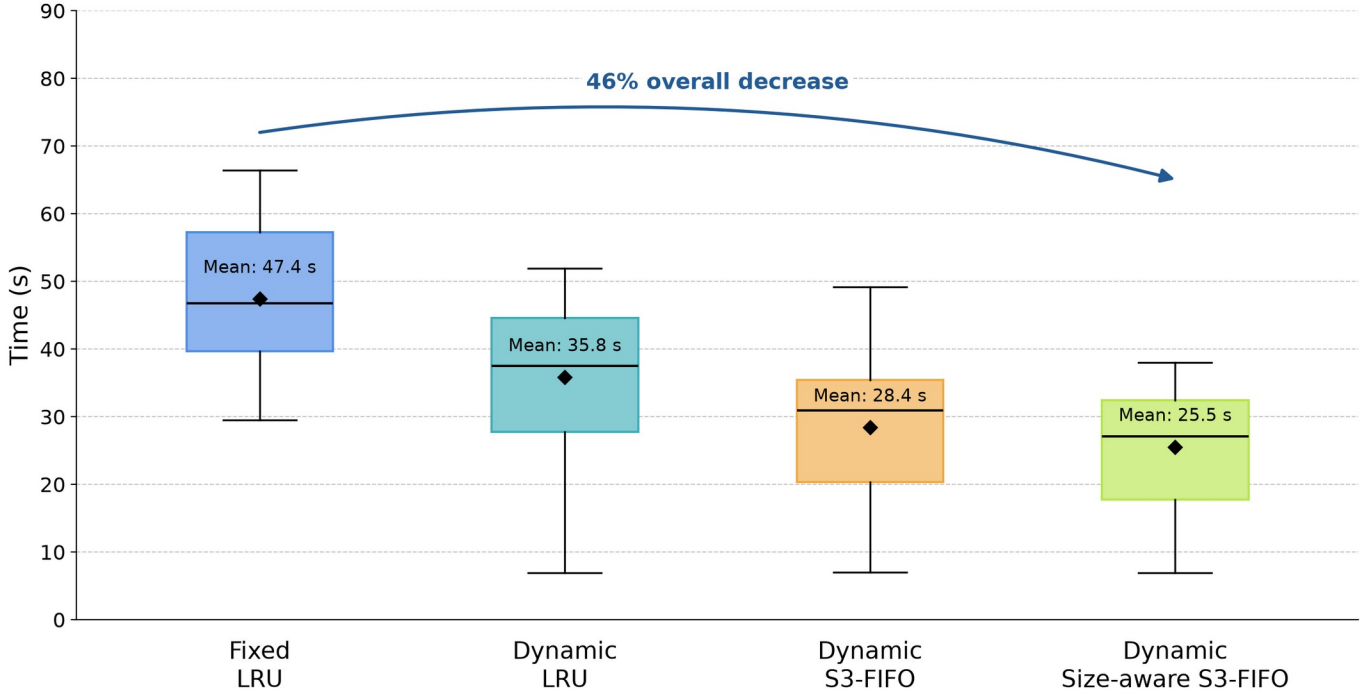
A scan-resistant policy can further improve performance

Size-aware Policy



Making S3-FIFO size-aware improves average and worst-case performance

Overall Comparison



Significant improvement on every query sequence

Summary

How can we design a cache optimized for analytical workloads in cloud environments?

- **Column-chunk caching** aligns cache granularity with the file layout, reducing unnecessary data cached
- **Scan-resistant policy** prevents cache pollution on long scans
- **Size-aware replacement** improves cache performance with variable-sized entries
- **Combined approach** outperforms fixed-size caching with LRU for the evaluated workloads